🧊 Rubik's Cube Solver - Interview Preparation Guide

This guide breaks down your project end-to-end to help you ace your interview tomorrow. It covers the core architecture, the rationale behind your technology choices, how it all works under the hood, and how to defend your decisions against alternatives.

---

1. Project Overview & Architecture

At a high level, your app is a native Android application that allows users to scan a scrambled Rubik's Cube using their smartphone camera. It parses the colors of the 54 facelets (9 on each of the 6 faces), translates those colors into a mathematical cube state, and computes a near-optimal sequence of moves to solve it.

The architecture is broken down into three main layers:

1. **Camera/UI Layer:** Uses **CameraX** to stream live frames to the screen.
2. **Vision/Processing Layer:** Uses a custom **HSV-based color classifier** to sample pixels and determine the color of each sticker.
3. **Algorithmic Solver Layer:** Uses **Kociemba's Two-Phase Algorithm** (a highly optimized Java implementation) to calculate the solution.

---

2. Technology Deep Dive: How it works & Why you chose it

A. Camera system: AndroidX CameraX

- **How it works:** CameraX binds to the lifecycle of your `MainActivity` or Fragment. You set up an `ImageAnalysis` use case that feeds frames from the camera directly into your vision logic (`CubeColorClassifier`) and a `Preview` usecase that displays the camera feed on the screen with the `ScanOverlayView`.
- **Why it's used:** It abstracts away the massive complexity of the native `Camera2` API. It automatically handles device-specific quirks (like aspect ratios and rotation) and cleans up resources automatically when the app is backgrounded, preventing memory leaks.
- **Why it's better than alternatives:**
  - *vs. Camera2 API:* Camera2 requires hundreds of lines of boilerplate code just to open a camera and manage state. CameraX is concise and lifecycle-aware.
  - *vs. OpenCV Camera:* OpenCV brings a massive C++ dependency to the app, increasing the APK size significantly. CameraX keeps the app lightweight.

B. Computer Vision: Custom HSV Color Classification

- **How it works:** When a frame is captured, your app isolates the 3x3 grid area. Instead of checking the whole square (which might have black sticker borders or glare on the edges), it samples only the **inner 40%** of each cell (30% to 70% bounds). It averages the RGB pixels and converts them to **HSV (Hue, Saturation, Value)**. It then compares this to 6 predefined reference colors using a weighted Euclidean distance where **Hue is weighted heavily (2.0x)** and Value/Brightness is weighted less (0.5x).

- **Why it's used:** If you try to compare colors using standard RGB, a shadow falling on a Red sticker might make it look dark brown, and a glare on an Orange sticker might make it look White or Yellow. **HSV separates color (Hue) from lighting (Value)**. By weighting Hue heavily, the app reliably detects the true color regardless of room lighting.
- **Why it's better than alternatives:**
  - *vs. RGB Comparison:* RGB is heavily impacted by shadows and glare, leading to misclassifications.
  - *Better Option to mention (Machine Learning):* A superior, but more complex alternative would be using **TensorFlow Lite** to run a CNN (Convolutional Neural Network) or YOLO model to perform semantic segmentation. An ML model could learn to ignore glare completely. You can mention this as a "future improvement" in the interview to show forward-thinking.

C. Solver Engine: Kociemba's Two-Phase Algorithm

- **How it works:** A Rubik's cube has 43 quintillion possible states. Searching blindly is impossible. Kociemba breaks the solve into two phases:
  - **Phase 1:** Restricts the cube so that corners can't be twisted and edges can't be flipped. It essentially puts the cube into a specific subgroup of states (using a pre-computed lookup table).
  - **Phase 2:** Solves the rest of the cube using only double turns for certain faces (`U, D, R2, L2, F2, B2`).
- **Why it's used:** It guarantees a solution in **20 moves or fewer** (God's Number) and computes it almost instantly (usually in milliseconds) on a mobile processor. Your specific implementation utilizes symmetries to compress the lookup tables down to just ~1MB of memory footprint.
- **Why it's better than alternatives:**
  - *vs. Beginner's Method (Layer-by-Layer):* Beginner algorithms take 60-100 moves to solve. It's too slow and tedious for a computer app to display to a user.
  - *vs. Korf's Algorithm (A* Search)🤫 Korf's always finds the *absolute optimal* fewest moves, but it can take hours to compute on a phone and uses massive amounts of RAM. Kociemba is the perfect balance: near-optimal (<=20 moves) and lightning-fast.

D. Language: Java

- **Why it's used:** The algorithm requires highly optimized, low-level bitwise operations and array manipulations. Your `Search.java` and `CoordCube.java` classes are heavily optimized. Furthermore, a lot of the best open-source implementations for Kociemba are written in Java.
- **Better Option to mention (Kotlin / C++):**
  - *Kotlin:* It is the modern standard for Android. You can mention that migrating the UI/Camera layers to Kotlin for null-safety and Coroutines (for background processing) would be a great next step.
  - *C++ (JNI):* For raw performance, moving the computer vision and Kociemba solver into C++ using the Android NDK would make the app incredibly fast, though it increases code complexity.

---

3. How to pitch the Competitive Advantage

If the interviewer asks, "*Why is this better than other Rubik's Cube solvers out there?*", you should highlight **Efficiency and Independence**:

1. **Zero Bloat / Lightweight:** Many scanner apps rely on importing massive OpenCV libraries (adding 20-40MB to the app size). Your app does color classification purely using mathematical distance on the native Android `android.graphics.Bitmap` and HSV conversions.
2. **Lighting Resilient Algorithm:** Relying on the weighted HSV algorithm rather than RGB means the user doesn't need perfect studio lighting to scan their cube.
3. **Memory Efficient Solver:** The Kociemba implementation utilizes coordinate symmetries to keep the massive lookup tables under 1MB. It runs entirely on-device (no cloud API needed), meaning it works offline, instantly, and doesn't drain the battery.